

JavaScript Closures - Complete Guide

What is a Closure?

A closure is a function that has access to variables from its outer function scope, even after the outer function has finished executing. This is one of the most powerful concepts in JavaScript!

Basic Example:

```
function outer() {  
  let count = 0; // outer variable  
  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = outer();  
console.log(counter()); // 1  
console.log(counter()); // 2  
console.log(counter()); // 3
```

Why Closures Are Useful:

1. Data Privacy - Variables can't be accessed directly
2. State Management - Keep track of data between calls
3. Encapsulation - Hide implementation details

Real World Example: Bank Account

```
function createAccount(initialBalance) {  
  let balance = initialBalance; // PRIVATE  
  
  return {  
    deposit: (amount) => { balance += amount; },  
    withdraw: (amount) => { balance -= amount; },  
  };  
}
```

```
getBalance: () => balance
};
}

const myAccount = createAccount(1000);
myAccount.deposit(500); // balance = 1500
myAccount.withdraw(200); // balance = 1300
console.log(myAccount.getBalance()); // 1300
// myAccount.balance = 9999; // ERROR! Can't access directly
```

Important Point:

The 'balance' variable is protected inside the closure. It can ONLY be accessed through the returned methods (deposit, withdraw, getBalance). This is data privacy!